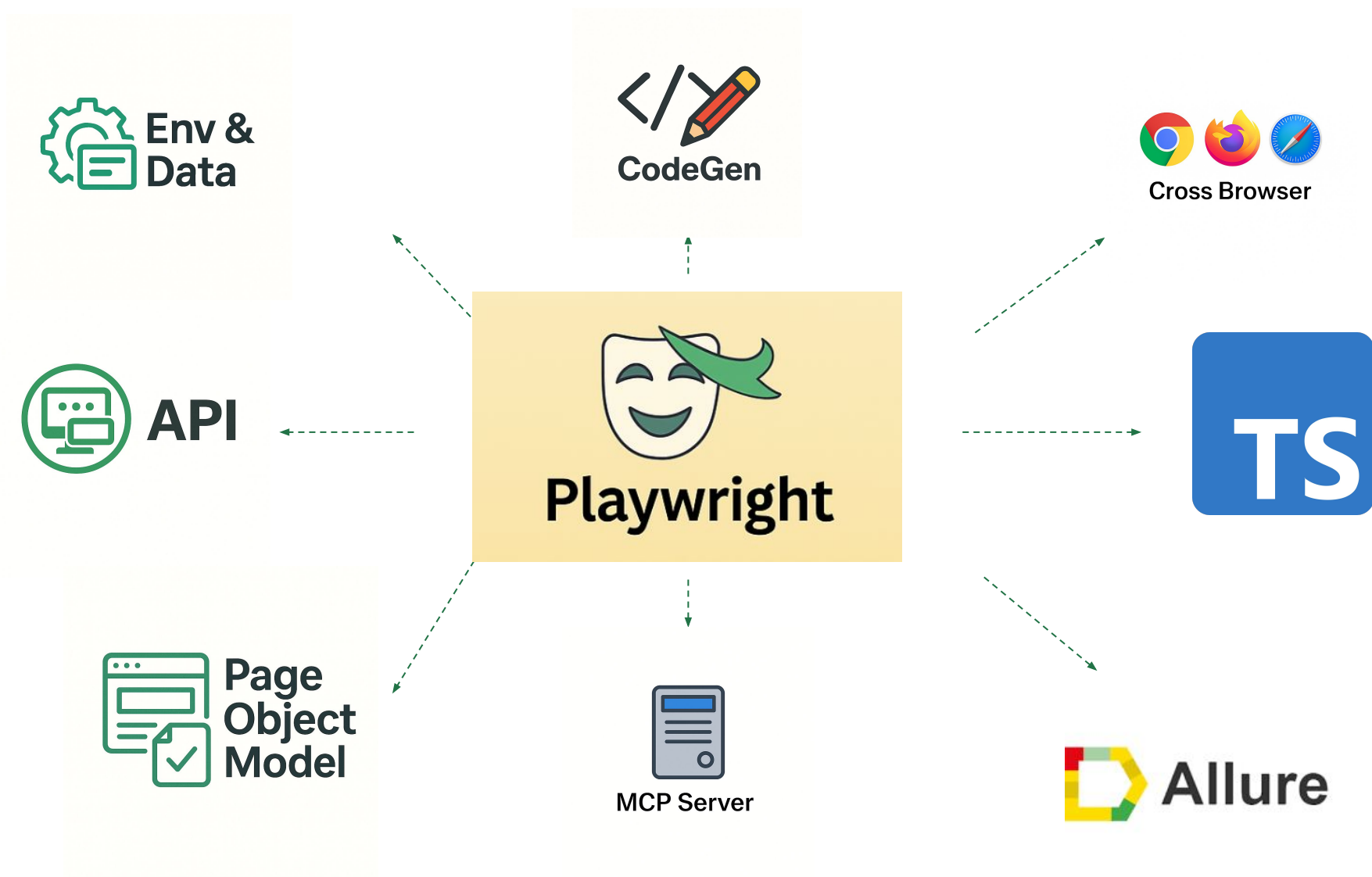


Playwright – E2E Automation Framework

Cope Automation

Playwright Framework Components

👋 Playwright – E2E Test Framework Components



Playwright Overview

What is Playwright?

“A **person** who writes plays.”

Why Playwright?

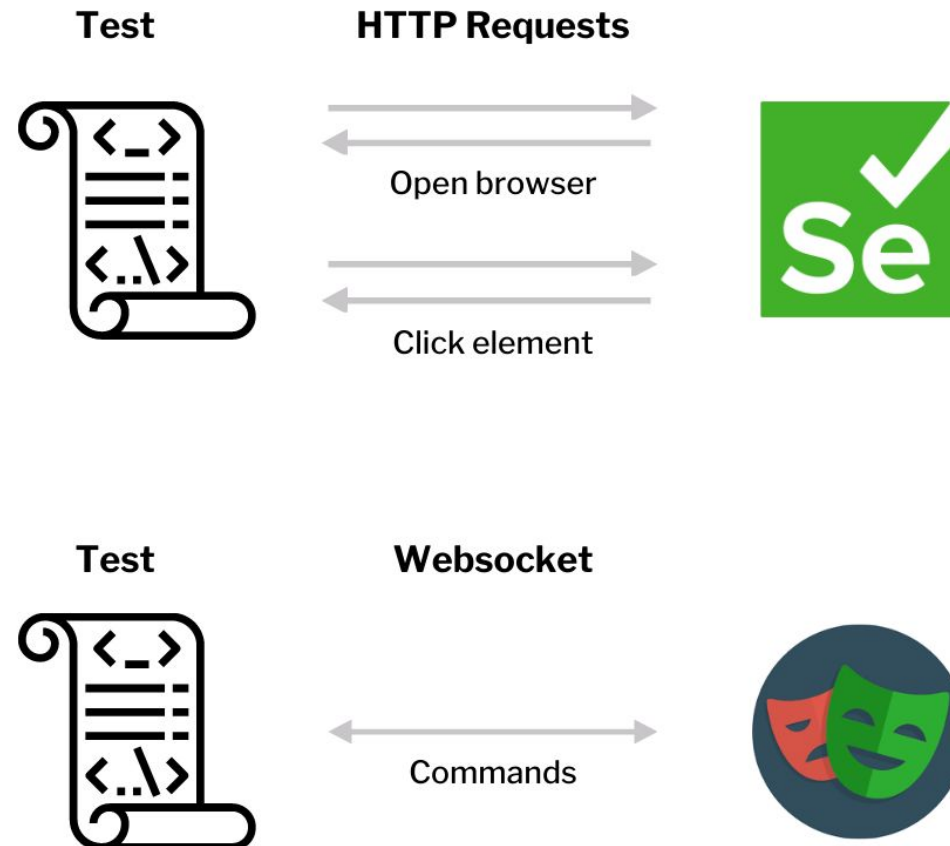
Top 5 Reasons

1. 3x cross: Platform, Browsers (And Devices), Language
2. Own Built-in test runner with inbuilt Assertions
3. Rich features like auto-waiting, code-gen, annotations, tags, many reporters
4. Integrated API Testing
5. Powerful MCP server - backed up Microsoft

Playwright Architecture

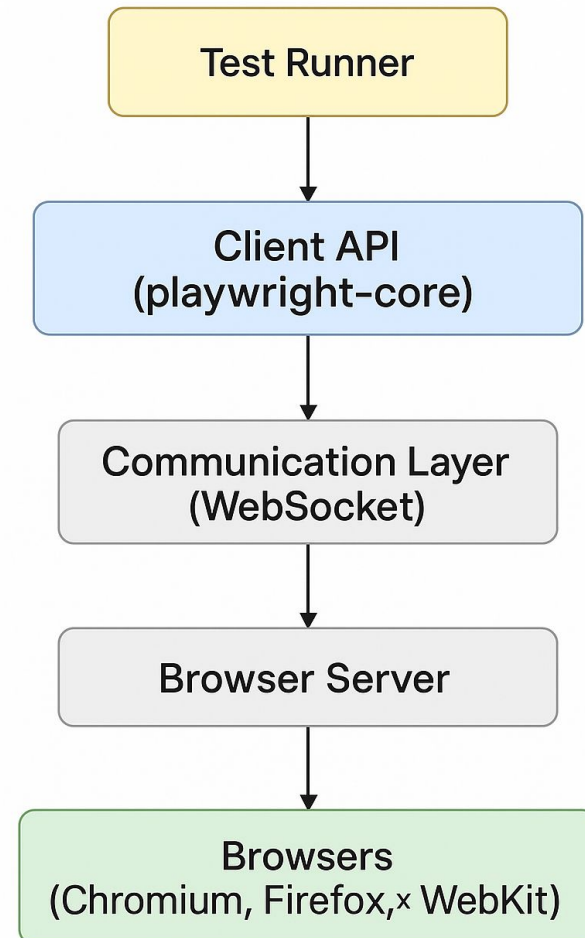
Playwright Architecture

Playwright - the communication happens via secure **websocket** protocol



Playwright Test Runner

Playwright Architecture



Format of the Course

What is my commitment to the course?

My commitments

1. Keep the content up-to-date as much as possible
2. Responding to your queries as quickly as I can
3. Providing additional resources like (.md file, code reference)

What do I expect from you?

What can you do for me?

1. Your time commitment to the learn and implement

2. Provide enough information when raising questions

3. Feedback

Thank you !

Let's get started 

Playwright Introduction

Installation

What is Node.js ?

It's a ***runtime environment*** for running JavaScript code.
With a TypeScript compiler, it runs TypeScript code as well.

Set up first node project

1. Create a folder (preferably under
`/documents/workspace/<name-of-the-project>`)
2. Initialize node project
 - `npm init [-y]`
3. Check `package.json` is created
4. Run a simple JS code
5. Done ! 🎉

Best Practices - Naming Conventions

Naming Convention - Best Practice:

16.2. File & Code Naming Conventions

Item	Convention	Example
Folders / Files	kebab-case	<code>page-objects/</code> , <code>file-helper.ts</code>
Page and Spec files	dot-separated	<code>nopcommerce.home.page.ts</code> , <code>nopcommerce.e2e.spec.ts</code>
Class Names	PascalCase (each word capitalized)	<code>LoginPage</code> , <code>DashboardActions</code>
Variables	camelCase	<code>loginButton</code> , <code>userNameInput</code>
Constants	UPPER_SNAKE_CASE	<code>BASE_URL</code> , <code>API_TIMEOUT_MS</code>

 Consistent naming improves readability and reduces confusion across teams.

GitHub Integration

Git - Overview

“Git is a distributed **version control** software/system, It is a free and open source project.”

What is it used for?

1. Version controlling the code

2. Branching Strategy (feature, test, prod, master)

3. Team Collaborations

4. Tracking Change History

5. CI/CD Integration

First Playwright



Did you know ?

What is Playwright?

“A person who writes plays.”



Playwright

Rules About Playwright Spec/Test file...

Playwright recognizes the following file extensions as **valid test specification files**

*.spec.ts

*.test.ts

Rules About Playwright Spec/Test file...

Playwright recognizes the following file extensions as **valid test specification files**

*.spec.ts

*.test.ts

Understanding Core Concepts

Playwright Test Runner

Key Notes on “Playwright Test Runner”

1. Own built-in Test Runner (No external runners like Mocha, Jest, Jasmine)

2. Powerful Config file (Controls overall test settings incl browsers, reporters, parallel runs)

3. **test**, **expect**, **request** some of the most used functions



🔥 Most Used Other Imports from Playwright Runner

```
``ts
export const chromium
export const firefox
export const webkit
export const selectors
export const devices
export const errors
export const request
export const _electron
export const _android
export const test
export const expect
export const defineConfig
export const mergeTests
export const mergeExpects
export default playwright.test;
``|
```

Playwright Test Function

Signature of *test()* function

Declares a test.

- `test(title, body)`
- `test(title, details, body)`

Signature of ***test()*** function

Declares a test.

- `test(title, body)`
- `test(title, details, body)`

@param `title` — Test title.

@param `details` — Additional test details.

@param `body`

Test body that takes one or two arguments: an object with fixtures and optional [TestInfo](#).

Signature of ***test()*** function

Declares a test.

Title as **string**

- `test(title, body)`
- `test(title, details, body)`

Signature of ***test()*** function

Declares a test.

Body as callback function {<fixture e.g. page>}, [testInfo]

- `test(title, body)`
- `test(title, details, body)`

Signature of *test()* function

Declares a test.

- `test(title, body)`
 - `test(title, details, body)`
- Details as {<additional details like tag>}
- 

Most common Body args

 A test body take one of the following args

- page
- context
- request

Page Fixtures

DRY Principle

DRY - Don't Repeat Yourself

Page - Fixture

A page is a **fixture** from the playwright runner's perspective, which is passed as an arg to the test object.

✓ Most Commonly Used Methods

Method	Description
<code>goto(url)</code>	Navigate to a URL
<code>locator(selector)</code>	Create a locator for actions/assertions
<code>getByRole()</code> , <code>getByText()</code> , <code>getByLabel()</code>	Semantic locators for accessibility-based selectors
<code>click(selector)</code>	Clicks on a matching element
<code>fill(selector, value)</code>	Fills an input field
<code>evaluate(fn, arg)</code>	Runs JavaScript in the browser context
<code>waitForURL(url)</code>	Wait for navigation to a specific URL
<code>screenshot()</code>	Capture a screenshot
<code>title()</code>	Get the page title
<code>url()</code>	Get the current URL

** Note: The **'locator'** and **'getBy*'** methods return a **locator** objects

Locator Objects

Locator / Selector Object

Locators are the central piece of Playwright's auto-waiting and retry-ability. In a nutshell, locators represent a way to find element(s) on the page at any moment. A locator can be created with the [page.locator\(\)](#) method.

✓ Most Commonly Used Methods

Method	Description
<code>click()</code>	Click on the located element
<code>fill(value)</code>	Fill an input or textarea with the specified value
<code>press(key)</code>	Press a keyboard key on the focused element
<code>getAttribute(name)</code>	Retrieve a specific attribute from the element
<code>textContent()</code>	Retrieve the raw text content of the element
<code>isVisible()</code>	Check if the element is visible
<code>isHidden()</code>	Check if the element is hidden
<code>isChecked()</code>	Check if the element is selected (for checkbox/radio)
<code>check() / uncheck()</code>	Set checkbox/radio to checked or unchecked
<code>selectOption()</code>	Select an option from a dropdown
<code>evaluate(fn)</code>	Execute JS in the page context for the matched element
<code>screenshot()</code>	Take a screenshot of the matched element

Expect Function

Expect Function

What it's signature?

ts

```
expect(actual: any): Matchers<unknown>;
```

How to read the above signature?

The expect method takes any type of **'value'** (e.g. a page, locator, a number, string, object, array) and then **returns a Matchers**.

Depends on the return type of the matchers, the assertions methods can be used.

The “await” keyword explained!

Where to be awaited?

The **await** keyword is used in JavaScript and TypeScript to **pause the execution** of an **async** function until a **Promise** is resolved (success) or rejected (error). It helps you write asynchronous code that looks and behaves like synchronous code.

1. **await** before every “**page**” or “**locator**” action methods (e.g. `click()` and `fill()` ..etc)
2. **await** before “**expect**” methods

Codegen & Its usecases

Codegen - Overview

What it does?

1. Records test flow

2. Generates best locators

What are its benefits?

✓ Reduces test writing time drastically

✓ No more brittle/flaky selectors

✓ 'Record at cursor' - Enables adding locators in the middle of test flow



A big
productivity
Boost

Locator Strategies

Deep Dive into Playwright Locators

Key Takeaways

- ✓ `page.getBy*()` and `page.locator()` -> Returns `locator` objects
- ✓ The above methods not to be “**awaited**” as it does not return a promise
- ✓ The type of locator is an “object”
- ✓ Locators are LAZY



Codegen
helps find
the right
locator

Handling Web Elements

Element Type: Button/Link

Actions

1.Click

2.Press

3.Double click

4.Right Click

5.Hover

6.Add timeout if slow

Debugging Tests

Playwright Debugging Options

Options

1. VS Code Test Explorer

2. UI Mode (--ui)

3. Debug (PWDEBUG=1)

4. Trace Viewer

Allure Reporter

Allure Reporter

Why Allure Reporter

1. Rich UI

2. Frameworks Agnostic - Supports different frameworks like selenium, wdio and playwright

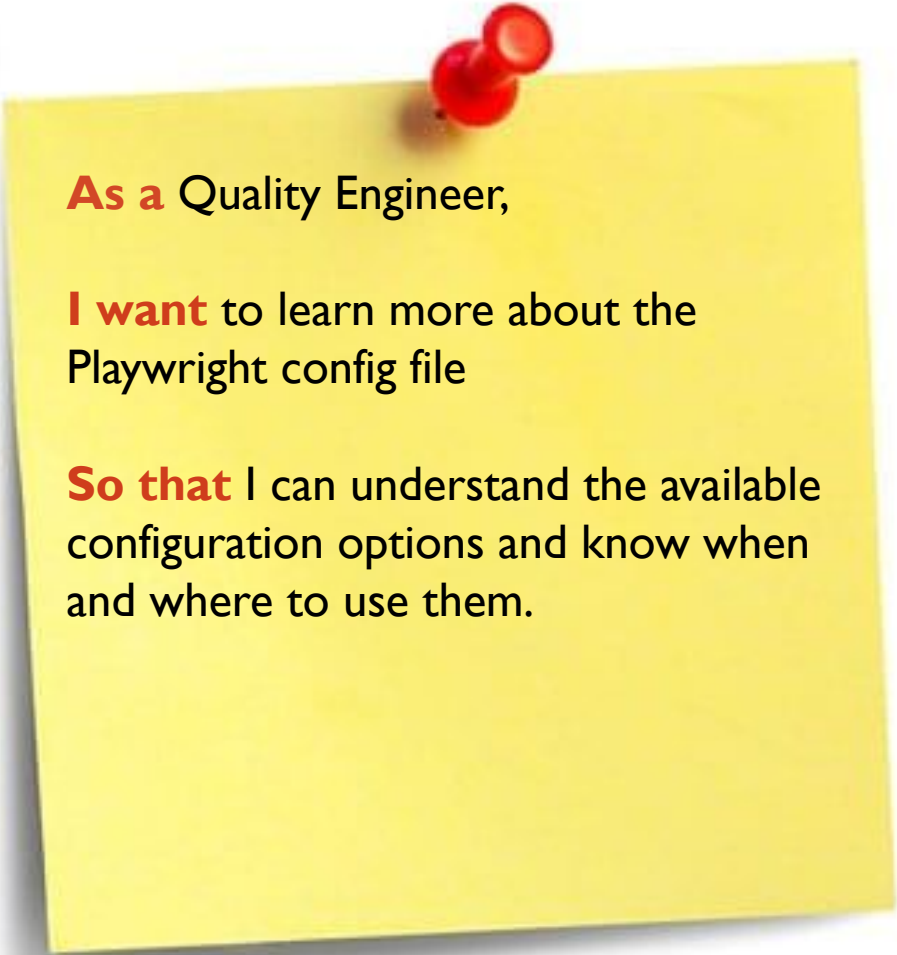
3. Detailed insights into Test Execution

4. Trend Tracking

5. Works in CI/CD

Playwright Core Capabilities

Test Config Overview

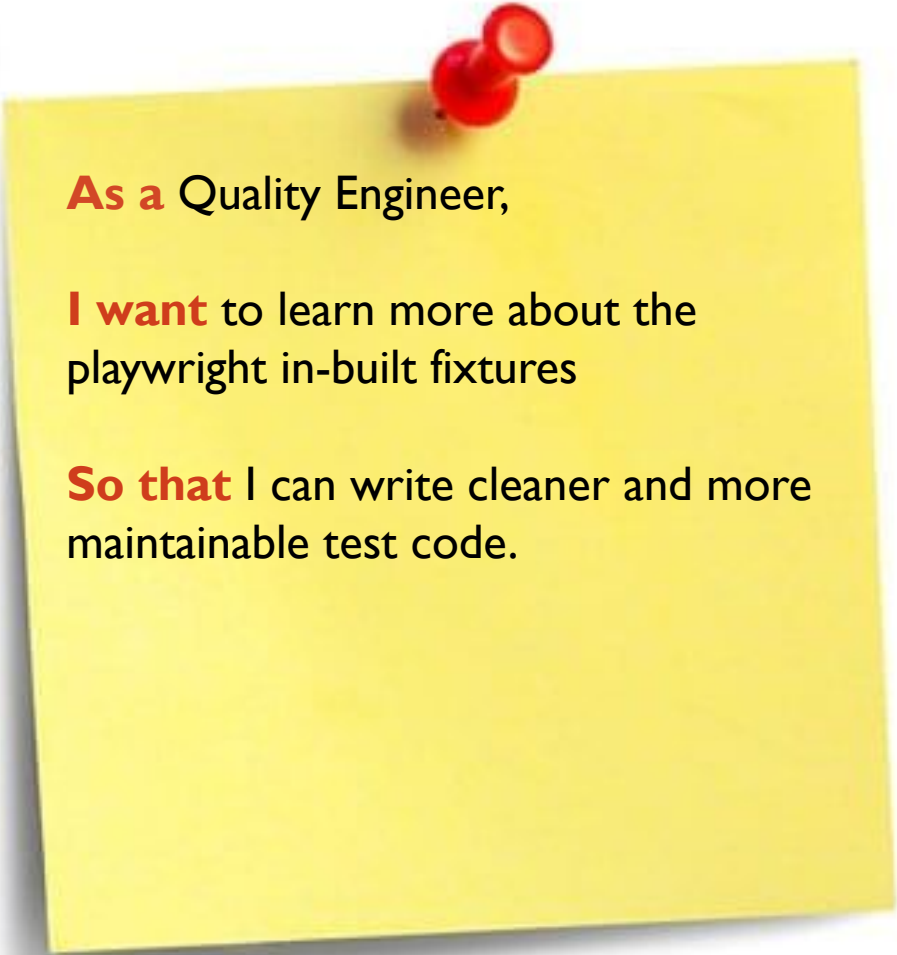


As a Quality Engineer,

I want to learn more about the
Playwright config file

So that I can understand the available
configuration options and know when
and where to use them.

Fixtures



As a Quality Engineer,

I want to learn more about the
playwright in-built fixtures

So that I can write cleaner and more
maintainable test code.

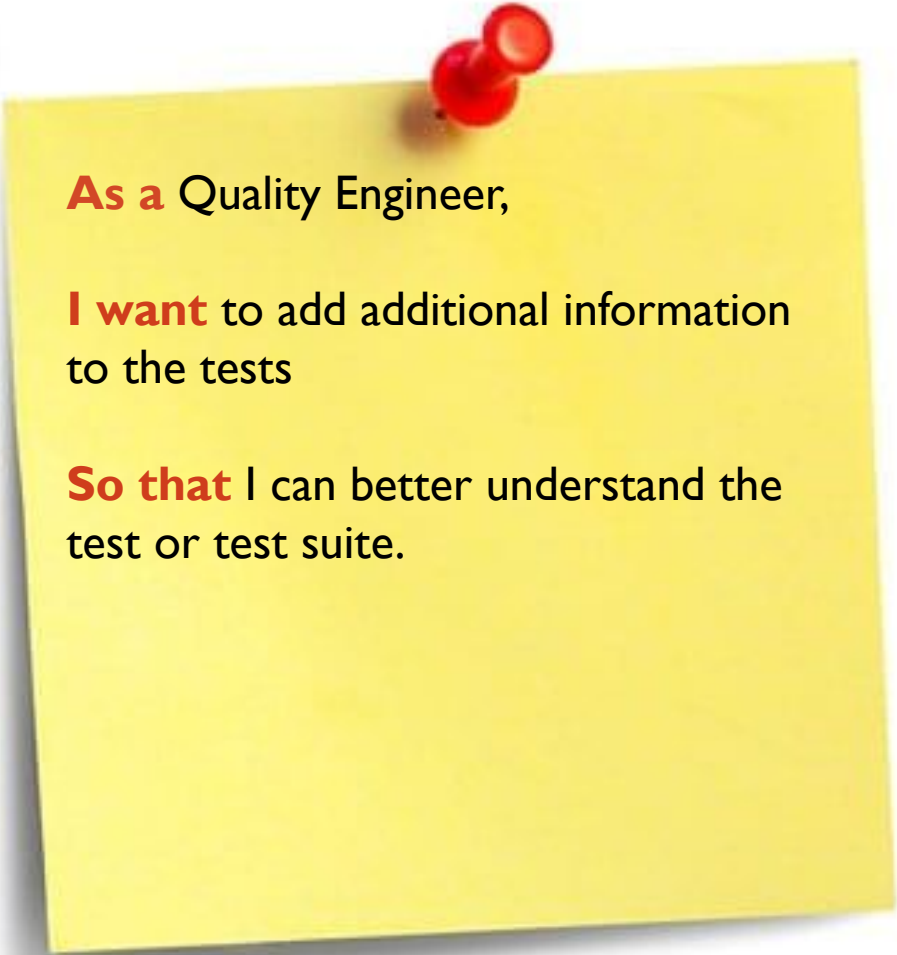
What is Fixture?

In simple terms, "**fixtures**" refers to **reusable** pieces of **setup or test context** that are required before running tests.

Fixtures solves one of the main problems of **DRY (Don't Repeat Yourself)**

*In other words, it's a **pre-built objects** loaded with actions*

Annotations

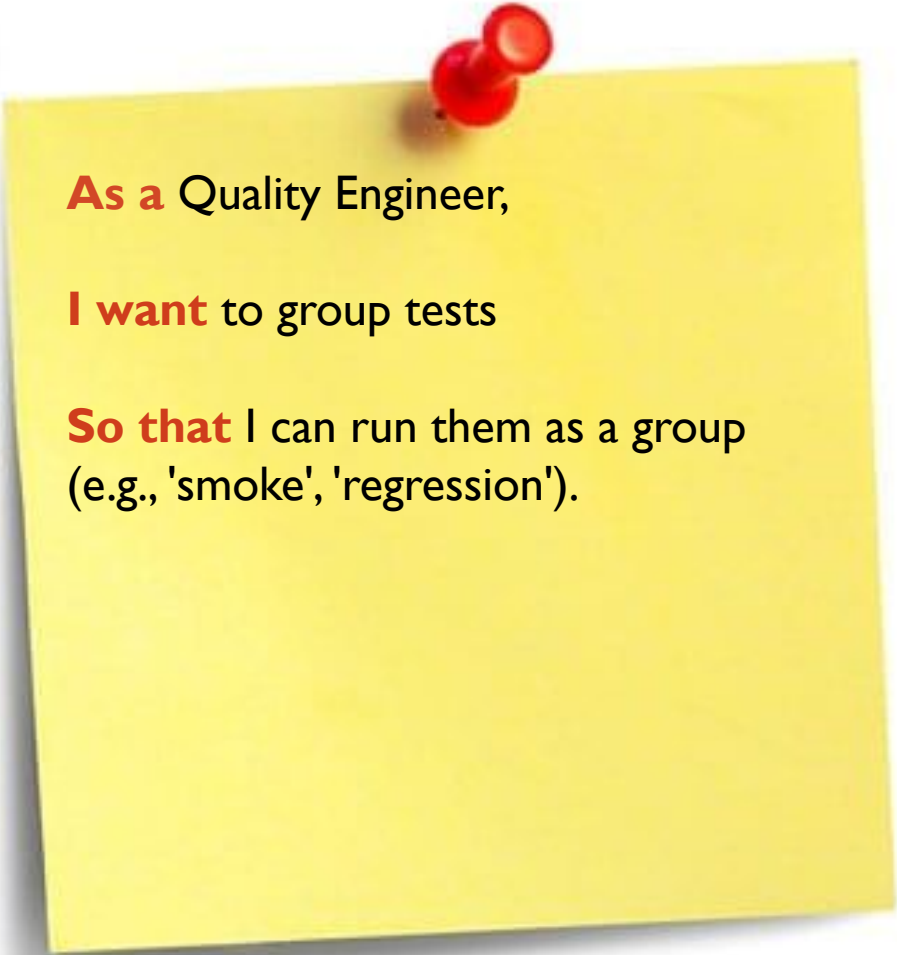


As a Quality Engineer,

I want to add additional information
to the tests

So that I can better understand the
test or test suite.

Tags



As a Quality Engineer,

I want to group tests

So that I can run them as a group
(e.g., 'smoke', 'regression').

Playwright Advanced Capabilities

🤔 Sounds familiar to you?

`implicitWait`

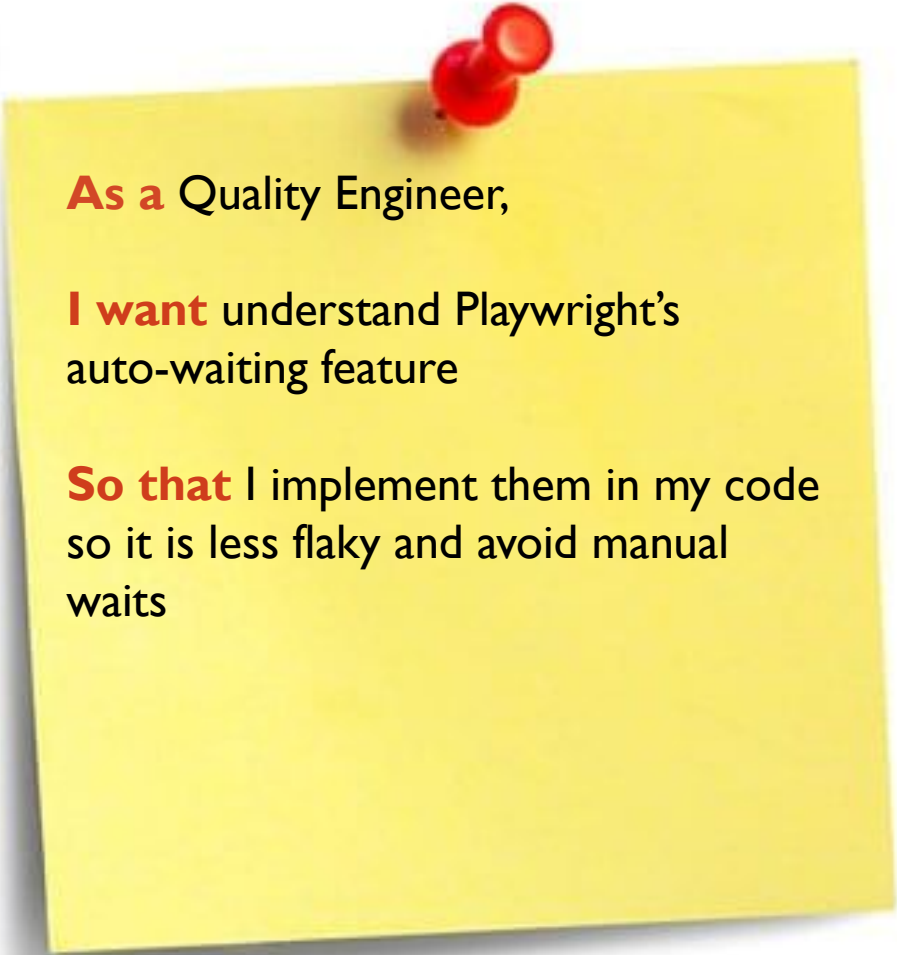
`explicitWait`

`sleep()`

`waitForDisplayed`

`pause()`

Auto-Waiting



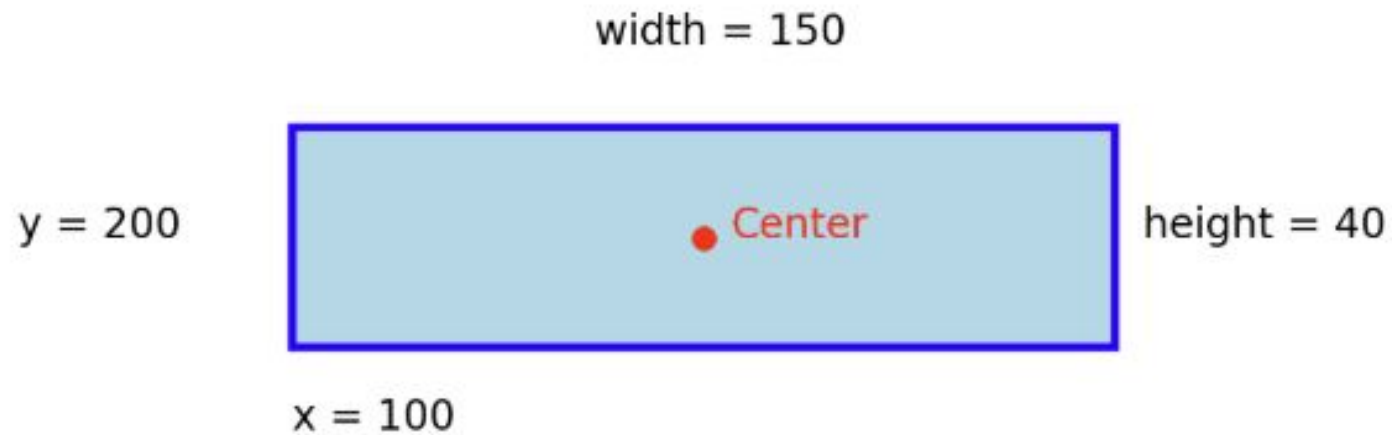
As a Quality Engineer,

I want understand Playwright's
auto-waiting feature

So that I implement them in my code
so it is less flaky and avoid manual
waits

Bounding Box

Bounding Box Example in Web Context





Timeout

As a Quality Engineer,

I want understand Playwright's
timeout setting

So that I set and configure the right
timeout for my test suite

*Order (highest → lowest):
Test-level → CLI → Config timeout*

Browser Args

As a Quality Engineer,

I want configure additional browser arguments

So that can overcome challenges when running tests in Chrome locally or in the CI environment.

As a Quality Engineer,

I want understand and use Playwright hooks

So that I can set up preconditions and clean up after my tests run in a structured way.

Hooks - Summary points

Remember

1. Hook can be at test file level or at `describe` level
2. The most common one being -> `describe` -> `beforeEach` -> `test`

Global Setup & Teardown

As a Quality Engineer,

I want to define **one-off** tasks such as setup and teardown activities

So that I have an ability to perform setup and teardown process in more controlled manner

[Mobile Web] Device Emulation

As a Quality Engineer,

I want execute tests across multiple **devices** and browsers

So that I can ensure the application works consistently across different environments.

Understanding Playwright Parallelism

As a Quality Engineer,

I want to run tests in parallel

So that I can reduce execution time
and achieve faster feedback.

Setting up 'tsconfig.json' file

As a Quality Engineer,

I want to maintain a TypeScript configuration file

So that I can customize project settings to align with specific requirements.

Understanding Data

What data goes where? - Organizing Data the Right Way

password

Test Data(data parameterization)

App URL

SQL Query Template

System Generated Auth
Token

API Endpoint(/user)

1.Static Data (constants.json)

2.Test Data (parameterization)

3.Dynamic Data (Global setup)

4.Env Config (config file)

5.Sensitive Data (.env)

Framework Building



Core Characteristics of Test Framework

Core Components

1.Modular

2. Maintainable

3. Extendable

4.Usable

5.Reporting

Page Object Model

As a Quality Engineer,

I want to set up a well-known Page Object Model (POM) design pattern

So that I can leverage its design principles to build a scalable and maintainable test framework.

Page Object Model

What's a page object?

A **page** or section of a web page as **an object**.

In technical terms, an object can have **properties (i.e elements) and methods (i.e. actions)**

API Testing Integration

Why API Testing?

Focus on the Logic, Not the UI

API testing verifies the **back-end logic** of an application by sending **HTTP requests** (GET, POST, PUT, DELETE) directly to endpoints — without using the front-end interface

HTTPS Methods

Common HTTP Methods Explained

Method	Purpose	Example
GET	Read/fetch data	<code>GET /students</code> → list all students
POST	Create a new item	<code>POST /students</code> with body <code>{name: "John"}</code>
PUT	Update an existing item	<code>PUT /students/123</code> with body <code>{name: "Johnny"}</code>
PATCH	Partially update an item	<code>PATCH /students/123</code> with <code>{age: 22}</code>
DELETE	Remove an item	<code>DELETE /students/123</code>

 All of these are just different **verbs** that describe what action you want to do with data on the server.

MCP Server Overview

What is MCP?

It is **Model Context Protocol (MCP)**

MCP is an open protocol that standardizes how applications provide context to LLMs.

Thank you!